

1. Project Title: AI-Proctored Online Examination Platform with Intelligent Performance Evaluation

2. Project Statement and Outcomes:

Conducting online examinations at scale demands far more than a basic quiz interface — the system must handle concurrent exam sessions without performance degradation, enforce strict proctoring to minimize academic dishonesty, support diverse question types including subjective written answers, and provide both automated and AI-assisted evaluation. This project builds a full-stack AI-proctored online examination platform using Python (FastAPI) and Next.js featuring a structured question bank (MCQ, multi-select, short answer, long answer, and image upload for handwritten answers), randomized paper generation, a timed exam engine with server-side session management and auto-submit on timeout, and an AI proctoring layer that uses computer vision and browser behavioural signals to detect and flag suspicious activity during live exam sessions. An LLM-based answer evaluation module provides AI-generated first-pass scoring for subjective answers, which examiners review and override, significantly reducing manual grading time.

The deliverable is a complete online examination platform with a FastAPI backend and a Next.js exam interface. Exam administrators create question banks across subjects, configure exam rules (duration, question count, randomization, negative marking), and schedule exam windows. Students receive a randomized exam paper on entry and are continuously monitored by the AI proctoring engine throughout the session: face presence verification, gaze direction tracking, multiple-person detection via webcam, and tab-switch / window-focus event logging are combined into a per-session suspicion score. MCQ and multi-select questions are auto-evaluated on submission; short and long answer questions are first scored by an LLM-based evaluation module and then queued for examiner review with the AI score and justification pre-filled. A results dashboard provides students with scores and question-level feedback. Examiners access a grading portal with AI-assisted scoring, annotation tools, and a proctoring review panel showing flagged session events with timestamps and webcam snapshots.

3. Modules to be Implemented:

- Question Bank Schema (MCQ + Subjective + Image), Auth & Exam Configuration
- Timed Exam Engine, Auto-Evaluation, AI Proctoring Engine & Subjective Grading Queue
- Next.js Exam Interface, AI Proctoring UI, Results & Examiner Grading Portal
- System Integration, Testing & Project Finalization

4. Week-wise Module Implementation and High-Level Requirements:

Weeks 1–2: Question Bank Schema (MCQ + Subjective + Image), Auth & Exam Configuration

- Design PostgreSQL schema for users (student, examiner, admin), question_bank (question_type enum: MCQ, multi_select, short_answer, long_answer, image_upload), options, exams, exam_questions, exam_sessions, answers, results, and proctor_events (webcam snapshots, gaze logs, tab-switch records per session) using SQLAlchemy ORM with Alembic migrations.
- Implement JWT authentication using FastAPI and python-jose with role separation: students (exam entry via secure token), examiners (question and grading management), and admins (platform configuration); implement exam access token generation with exam_id and student_id binding and short-lived session tokens to prevent token sharing during live exams.
- Build question bank management FastAPI endpoints for examiners: create questions of all types with subject tags, difficulty level, expected answer (for MCQ/multi-select), model answer text, marks, and negative mark configuration.
- Build the exam configuration API: examiners create exams specifying subject, question selection rules (count per difficulty, question type distribution), randomization mode (per-student unique paper), duration, start/end window, negative marking rules, and proctoring settings (enable/disable webcam monitoring, gaze tracking sensitivity threshold, maximum tab-switch warnings before auto-flag).
- Write pytest unit tests for question bank validation (MCQ must have exactly one correct option, image_upload questions must have max marks defined), exam configuration constraints, and randomized paper generation determinism per student ID.

Weeks 3-4: Timed Exam Engine, Auto-Evaluation, AI Proctoring Engine & Subjective Grading Queue

- Implement the timed exam session engine: on exam entry, generate a unique session token binding student_id to exam_id, record start timestamp, and enforce server-side time remaining calculation (client clock is never trusted); auto-submit when time expires via an APScheduler background job in FastAPI.
- Build answer submission APIs: students submit answers per question as they progress; MCQ/multi-select answers are recorded immediately; text answers are stored with word count validation; image upload answers are stored as files with a thumbnail generated server-side.
- Implement MCQ and multi-select auto-evaluation: on final submission, compare submitted option IDs against correct option IDs, calculate marks with negative marking deduction, and store per-question score in results table.

- Build the AI proctoring engine: (a) Face presence verification — use MediaPipe Face Detection (runs client-side in the browser via TensorFlow.js) to verify a face is present at exam start and continuously check throughout the session, flagging prolonged absence; (b) Gaze direction estimation — use MediaPipe FaceMesh landmarks to infer whether the student is looking at the screen or away, logging repeated off-screen gaze events; (c) Multiple-person detection — flag if more than one face appears in the webcam frame; (d) Browser behavioural signals — log tab-visibility-change events, window blur/focus events, and enforce single active session per student-exam pair. Combine all signals into a per-session suspicion score (0–100) stored in `proctor_events`, sent to the backend via a WebSocket heartbeat every 10 seconds.
- Build the subjective grading pipeline: after auto-evaluation of objective questions, pass all `short_answer` and `long_answer` submissions through an LLM-based evaluation module (OpenAI GPT-4o or equivalent) that generates a suggested score, a brief justification, and key points matched/missed against the model answer. `Image_upload` answers (handwritten) are routed directly to examiner review with OCR pre-processing (Tesseract or Google Vision API) to extract any legible text. All subjective answers are queued in the examiner grading portal with AI scores pre-filled and editable.

Weeks 5-6: Next.js Exam Interface, AI Proctoring UI, Results & Examiner Grading Portal

- Build a Next.js secure exam interface: question navigation panel (all questions, answered/flagged/unanswered status indicators), MCQ radio button UI, multi-select checkboxes, text area for short/long answers (with word count live counter), and a countdown timer with visual urgency states (amber at 5 min, red at 1 min). The exam session page must prevent right-click, text selection, and copy-paste via JavaScript event handlers.
- Implement image-based answer submission in Next.js: students can upload a photo of handwritten work or diagrams using a drag-and-drop image upload zone or device camera capture (MediaDevices API over HTTPS); preview the uploaded image before final submission.
- Build the AI proctoring UI in Next.js: integrate TensorFlow.js with MediaPipe Face Detection and FaceMesh running entirely client-side (no webcam frames are sent to the server — only derived signals). Display a small live webcam preview in the exam corner so students know they are being monitored. Show a warning modal on the first proctoring violation (face absent, multiple faces, or repeated gaze away) and increment the suspicion score on each subsequent event. Send the aggregated proctoring heartbeat payload to the FastAPI WebSocket endpoint every 10 seconds.
- Build a student results dashboard in Next.js: overall score summary, question-by-question result breakdown (correct/wrong/partial for MCQ; marks awarded by examiner for subjective), comparative percentile chart against the exam cohort,

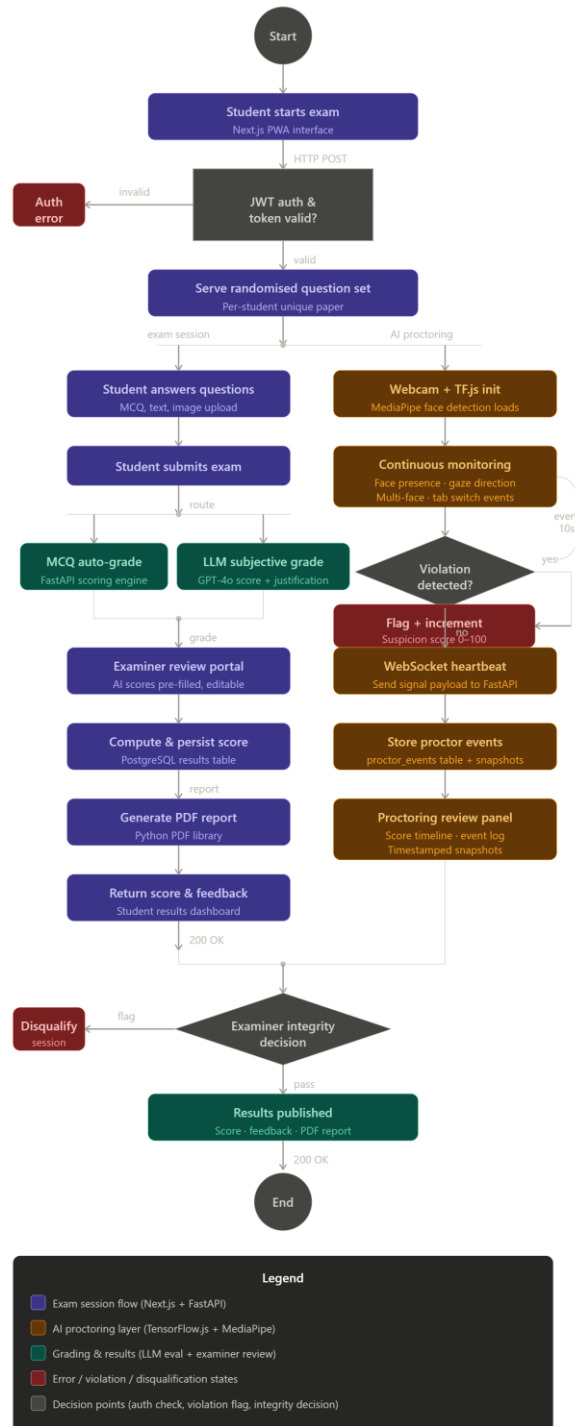
and question-level feedback from examiner annotations. Students do not see their proctoring suspicion score or event log.

- Develop an examiner grading portal in React: question-grouped answer review (all students' answers to the same question displayed sequentially for consistent grading), rubric-based marks input, annotation tools for image answers (highlight, text comment overlays), bulk grade submission, and results publication control.

Weeks 7-8: System Integration, Testing & Project Finalization

- Conduct end-to-end integration testing across all modules — question bank → exam configuration → timed session engine → AI proctoring engine → auto-evaluation → LLM grading → Next.js exam UI → image answers → results dashboard — and resolve any cross-module compatibility issues and edge-case failures.
- Perform load and stress testing on FastAPI endpoints using Locust or k6; validate AI proctoring accuracy by running simulated violation scenarios (covering camera, looking away, multiple faces) and confirming the suspicion score correctly reflects each event; conduct cross-browser UI validation on Chrome, Firefox, and Edge; address performance bottlenecks and implement enhancements from peer and mentor review.
- Refine the user experience based on testing feedback: improve interface clarity, add contextual help and input validations, ensure full responsiveness on mobile and tablet devices, and polish transitions and error states across all Next.js components. Specifically validate the proctoring UI: confirm the webcam preview does not obscure exam content, that violation warnings are clear but not disruptive, and that TensorFlow.js face detection does not cause noticeable frame drops on mid-range hardware.
- Prepare comprehensive project documentation covering system architecture, entity-relationship diagrams, API reference (FastAPI auto-generates Swagger/OpenAPI at /docs), deployment guide (Docker Compose for FastAPI + PostgreSQL + Next.js), database migration scripts (Alembic), AI proctoring model choices and limitations, and known limitations (browser permission requirements for webcam, TensorFlow.js performance on low-end devices, LLM evaluation accuracy caveats).
- Develop the final project presentation (PPT) covering problem statement, proposed solution, architecture, key features, technology stack, results, challenges, and future scope — and rehearse the full end-to-end demo for a smooth, confident delivery.

5. Flow chart:



6. Sample Dashboard:

